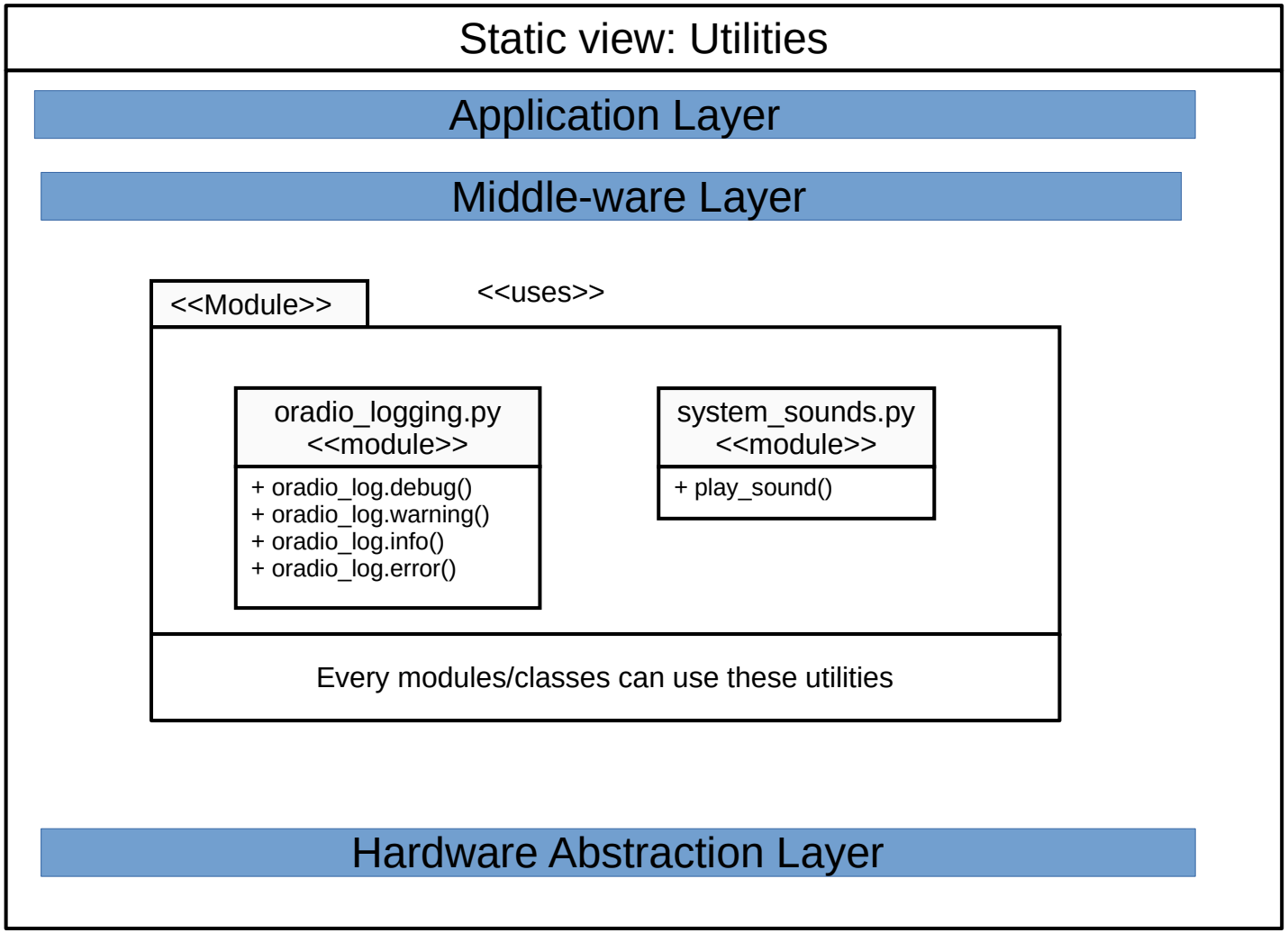
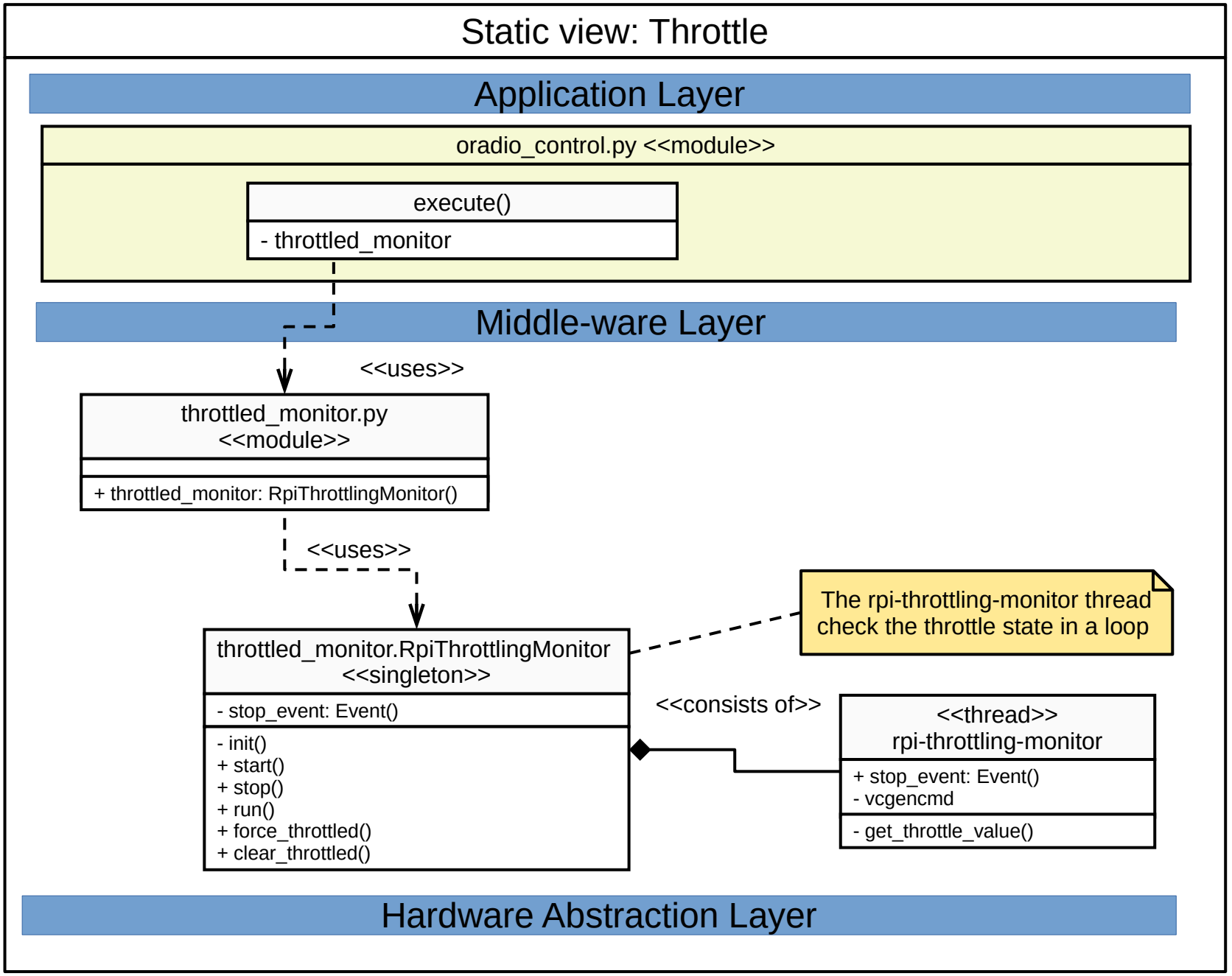

Static view: Utilities
Application Layer
Middle-ware Layer
<<Module>>
<<uses>>
oradio_logging.py
<<module>>
+ oradio_log.debug()
+ oradio_log.warning()
+ oradio_log.info()
+ oradio_log.error()
system_sounds.py
<<module>>
+ play_sound()
Every modules/classes can use these utilities
Hardware Abstraction Layer


Static view: Throttle
Application Layer
oradio_control.py <<module>>
execute()
- throttled_monitor
Middle-ware Layer
<<uses>>
throttled_monitor.py
<<module>>
+ throttled_monitor: RpiThrottlingMonitor()
<<uses>>
throttled_monitor.RpiThrottlingMonitor
<<singleton>>
- stop_event: Event()
- init()
+ start()
+ stop()
+ run()
+ force_throttled()
+ clear_throttled()
The rpi-throttling-monitor thread
check the throttle state in a loop
<<consists of>>
<<thread>>
rpi-throttling-monitor
+ stop_event: Event()
- vcgencmd
- get_throttle_value()
Hardware Abstraction Layer

Static view: Messages

Application Layer

oradio_control.py
<<module>>

+ execute(): void

Middle-ware Layer

<<class>>

volume_control.VolumeControl

spotify_connect.SpotifyConnect

web_service.WebService

usb_service.USBService

touch_buttons.TouchButtons

wifi_service.WifiService

- publish_command_message()

<<uses>>

<<Module>>

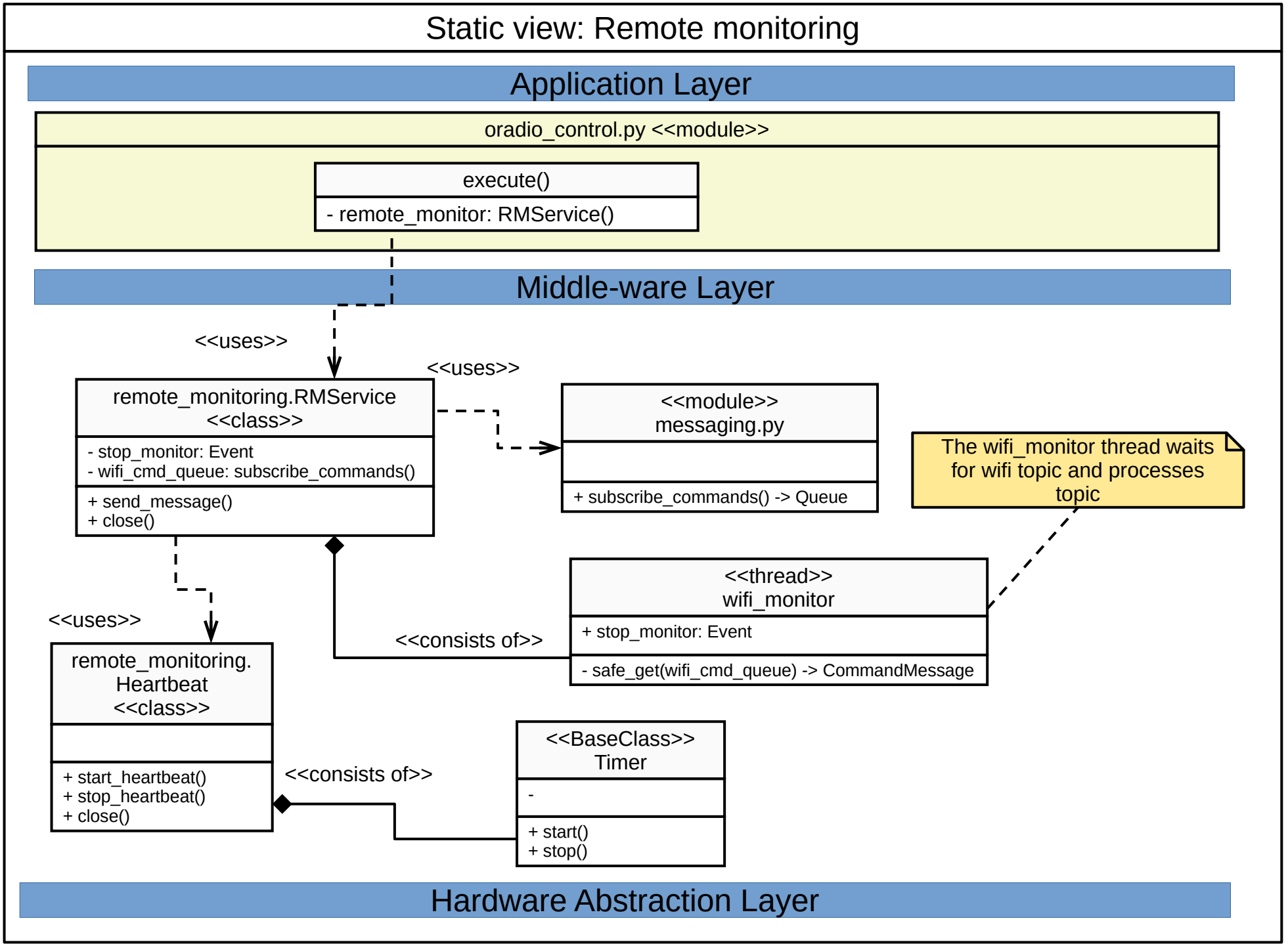
All module/classes can publish an error message

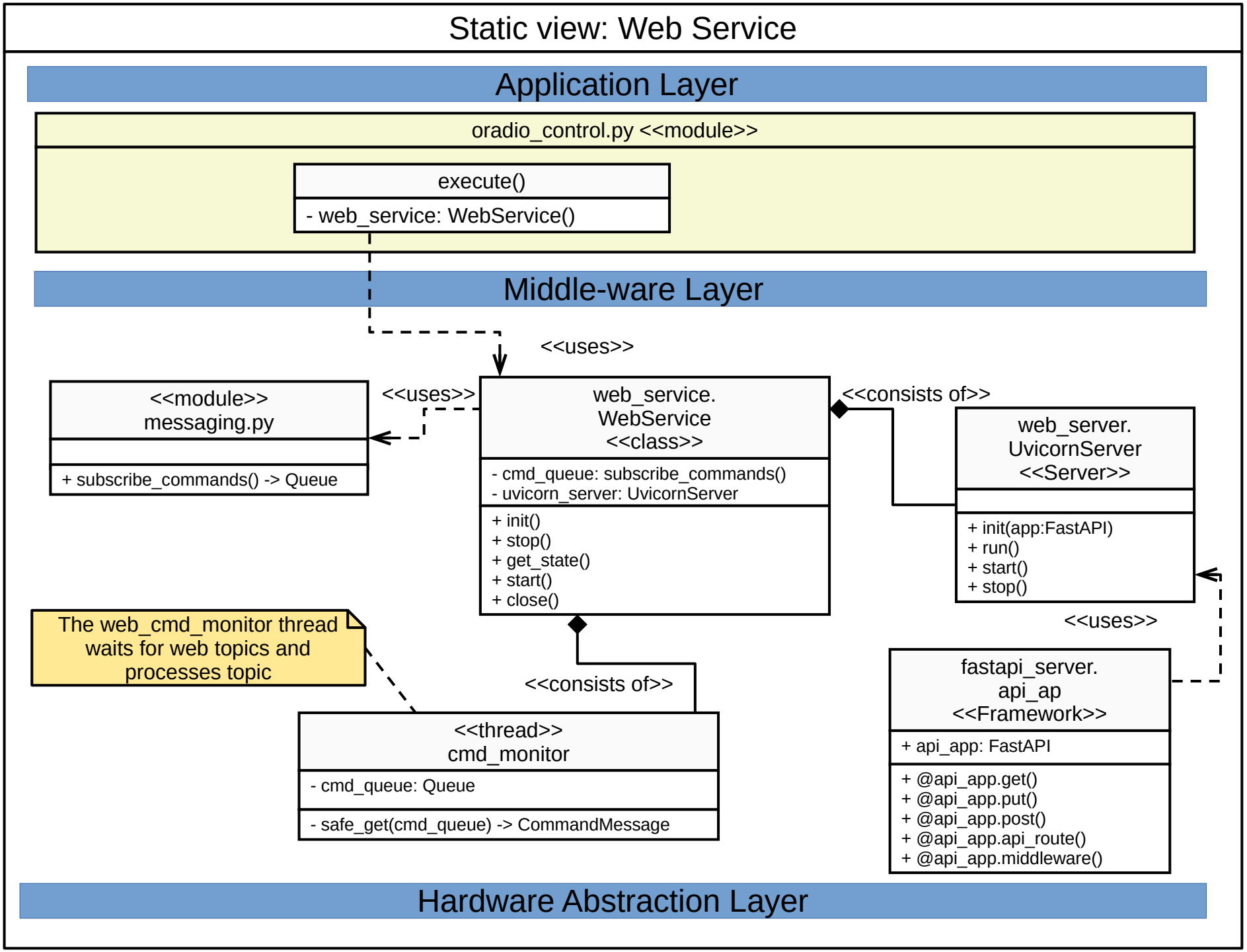
- publish_error_message()

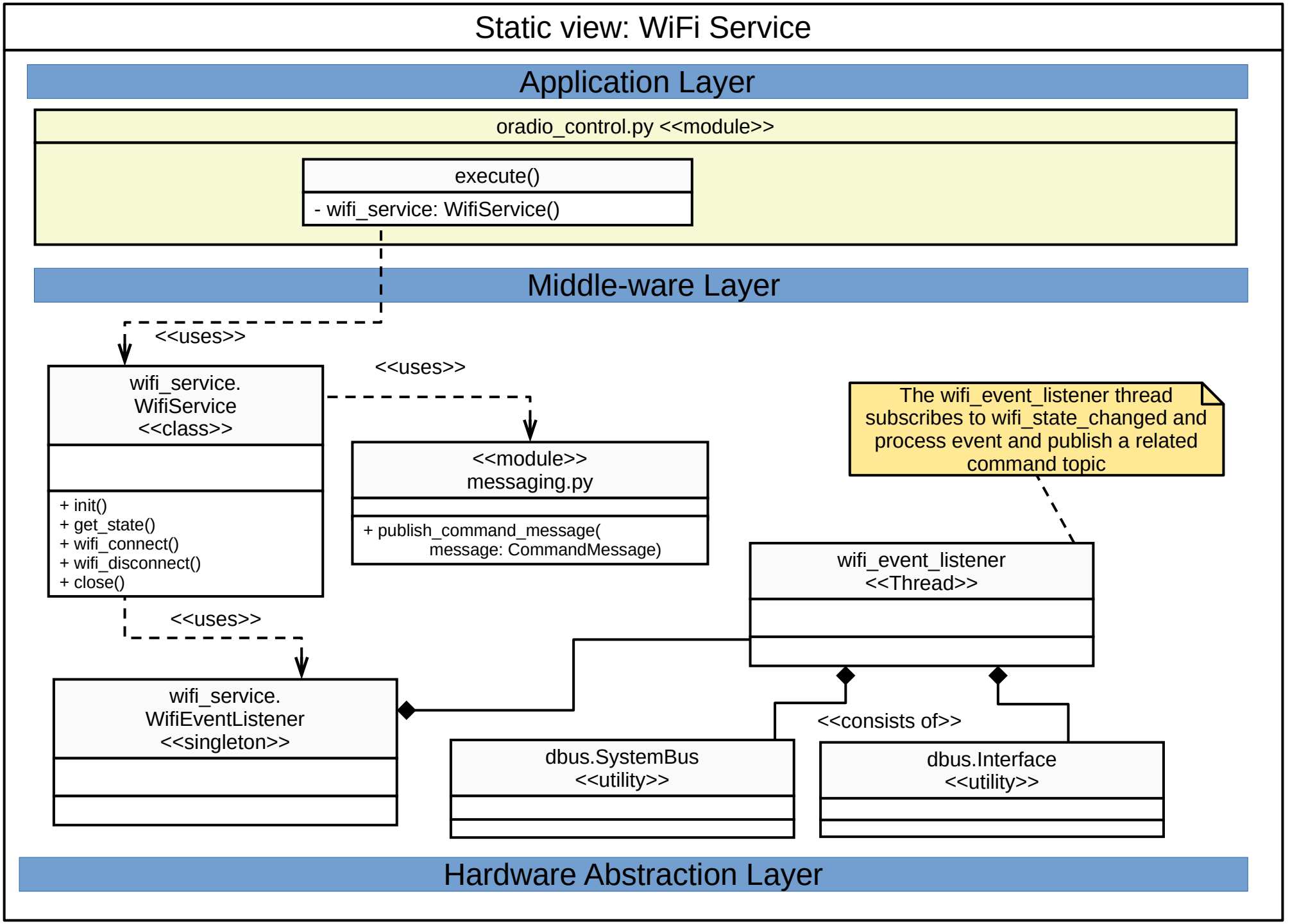
<<module>>
messaging.py

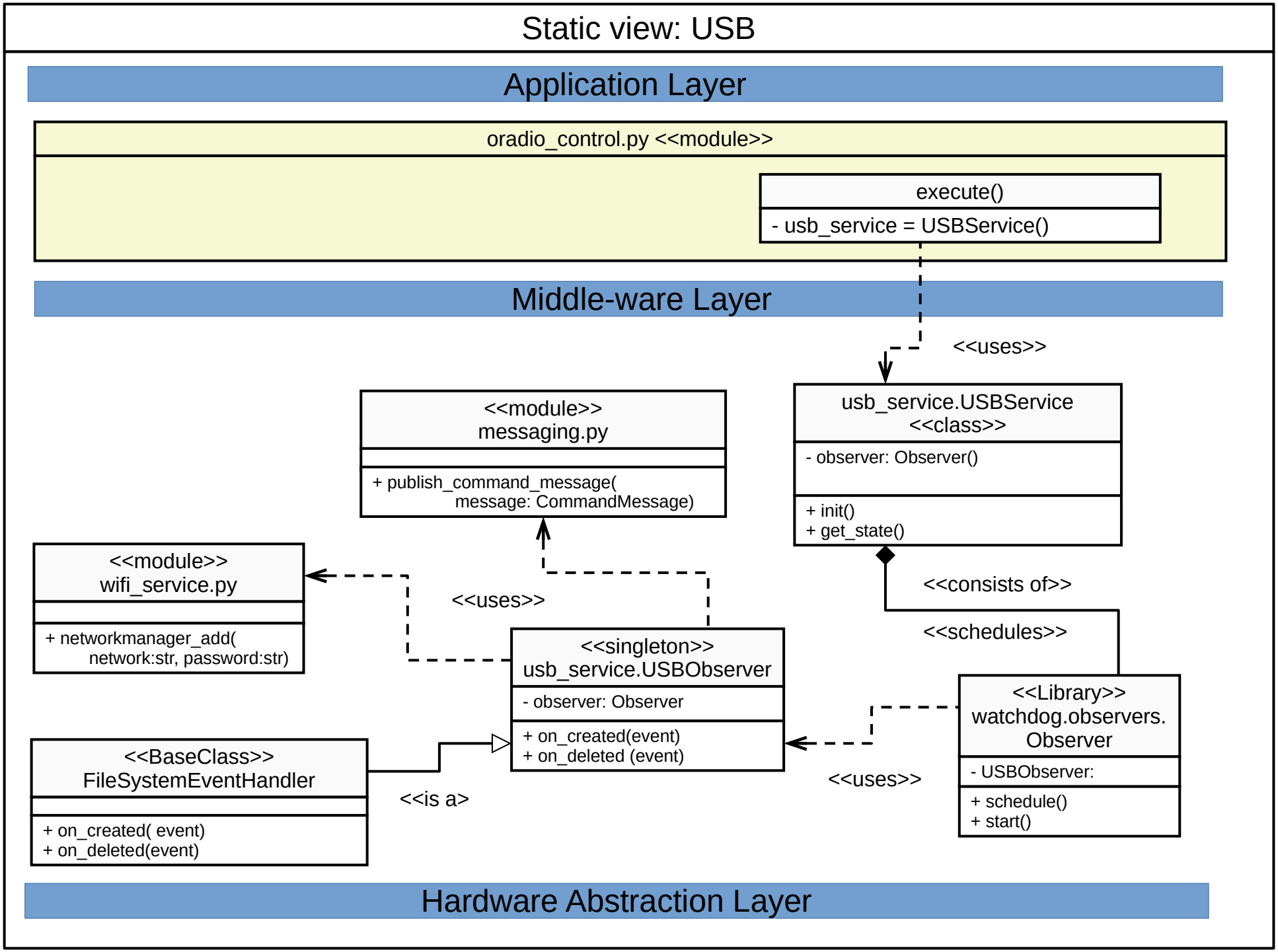
+ publish_command_message(message: CommandMessage)
+ publish_error_message(message: ErrorMessage)

Hardware Abstraction Layer

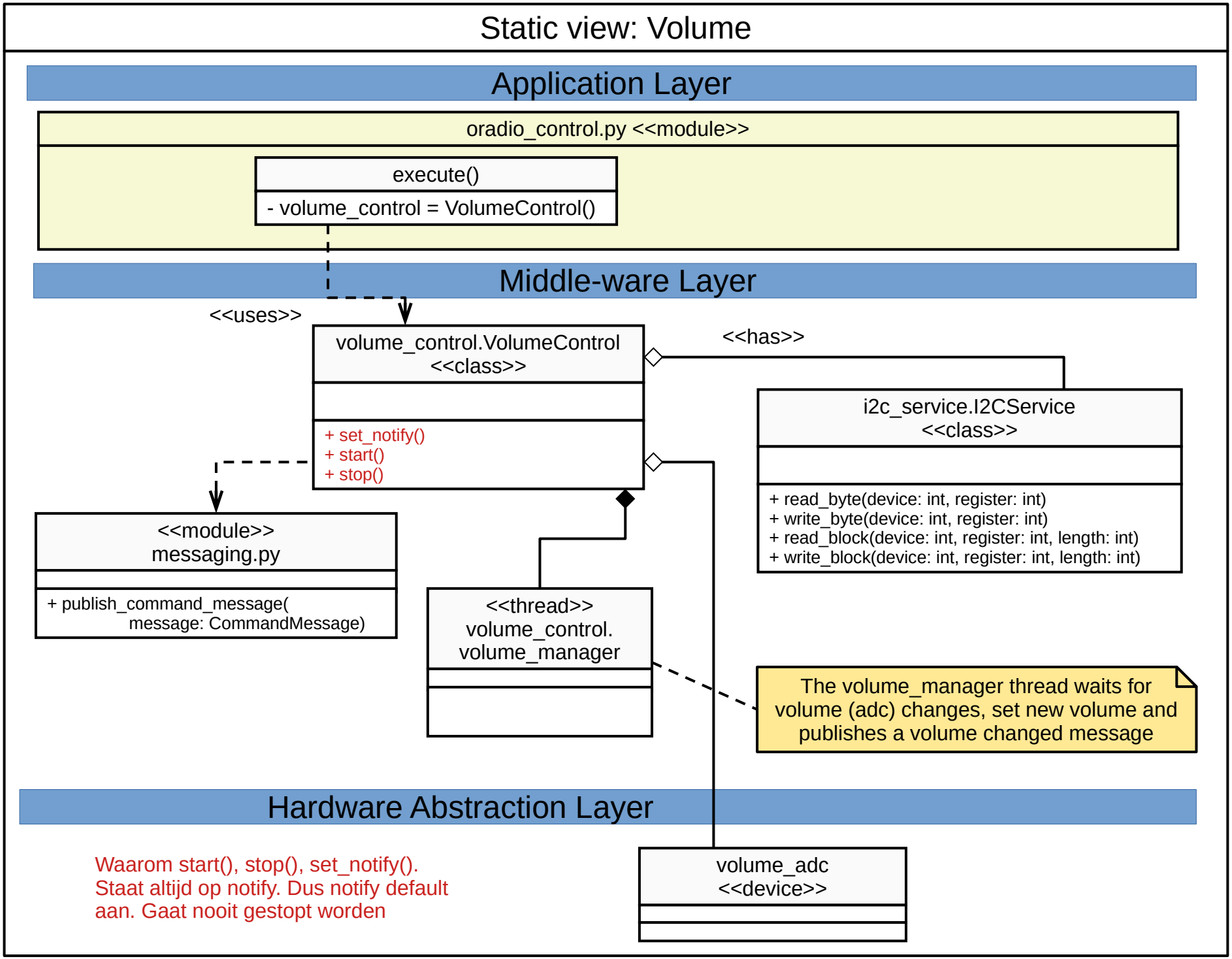

Static view: Remote monitoring
Application Layer
oradio_control.py <<module>>
execute()
- remote_monitor: RMService()
Middle-ware Layer
<<uses>>
<<uses>>
remote_monitoring.RMService
<<class>>
- stop_monitor: Event
- wifi_cmd_queue: subscribe_commands()
+ send_message()
+ close()
<<module>>
messaging.py
+ subscribe_commands() -> Queue
The wifi_monitor thread waits for wifi topic and processes topic
<<thread>>
wifi_monitor
+ stop_monitor: Event
- safe_get(wifi_cmd_queue) -> CommandMessage
<<uses>>
<<consists of>>
remote_monitoring.
Heartbeat
<<class>>
+ start_heartbeat()
+ stop_heartbeat()
+ close()
<<consists of>>
<<BaseClass>>
Timer
-
+ start()
+ stop()
Hardware Abstraction Layer

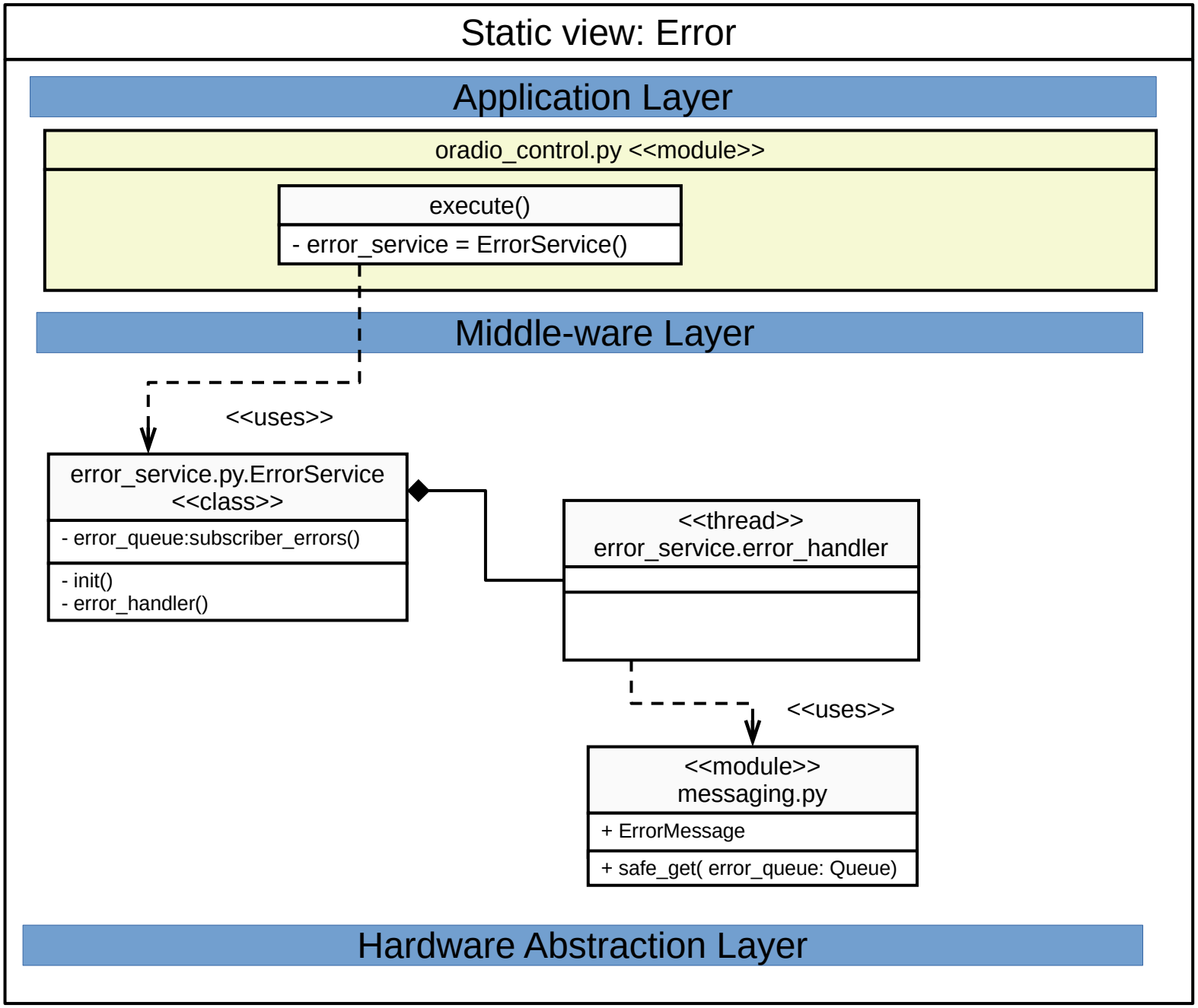

Static view: Web Service
Application Layer
oradio_control.py <<module>>
execute()
- web_service: WebService()
Middle-ware Layer
<<uses>>
<<module>>
messaging.py
+ subscribe_commands() -> Queue
<<uses>>
web_service.
WebService
<<class>>
- cmd_queue: subscribe_commands()
- uvicorn_server: UvicornServer
+ init()
+ stop()
+ get_state()
+ start()
+ close()
<<consists of>>
web_server.
UvicornServer
<<Server>>
+ init(app:FastAPI)
+ run()
+ start()
+ stop()
<<uses>>
The web_cmd_monitor thread
waits for web topics and
processes topic
<<consists of>>
<<thread>>
cmd_monitor
- cmd_queue: Queue
- safe_get(cmd_queue) -> CommandMessage
fastapi_server.
api_ap
<<Framework>>
+ api_app: FastAPI
+ @api_app.get()
+ @api_app.put()
+ @api_app.post()
+ @api_app.api_route()
+ @api_app.middleware()
Hardware Abstraction Layer


Static view: WiFi Service
Application Layer
oradio_control.py <<module>>
execute()
- wifi_service: WifiService()
Middle-ware Layer
<<uses>>
<<uses>>
wifi_service.
WifiService
<<class>>
+ init()
+ get_state()
+ wifi_connect()
+ wifi_disconnect()
+ close()
<<module>>
messaging.py
+ publish_command_message(
message: CommandMessage)
The wifi_event_listener thread subscribes to wifi_state_changed and process event and publish a related command topic
wifi_event_listener
<<Thread>>
<<uses>>
wifi_service.
WifiEventListener
<<singleton>>
<<consists of>>
dbus.SystemBus
<<utility>>
dbus.Interface
<<utility>>
Hardware Abstraction Layer

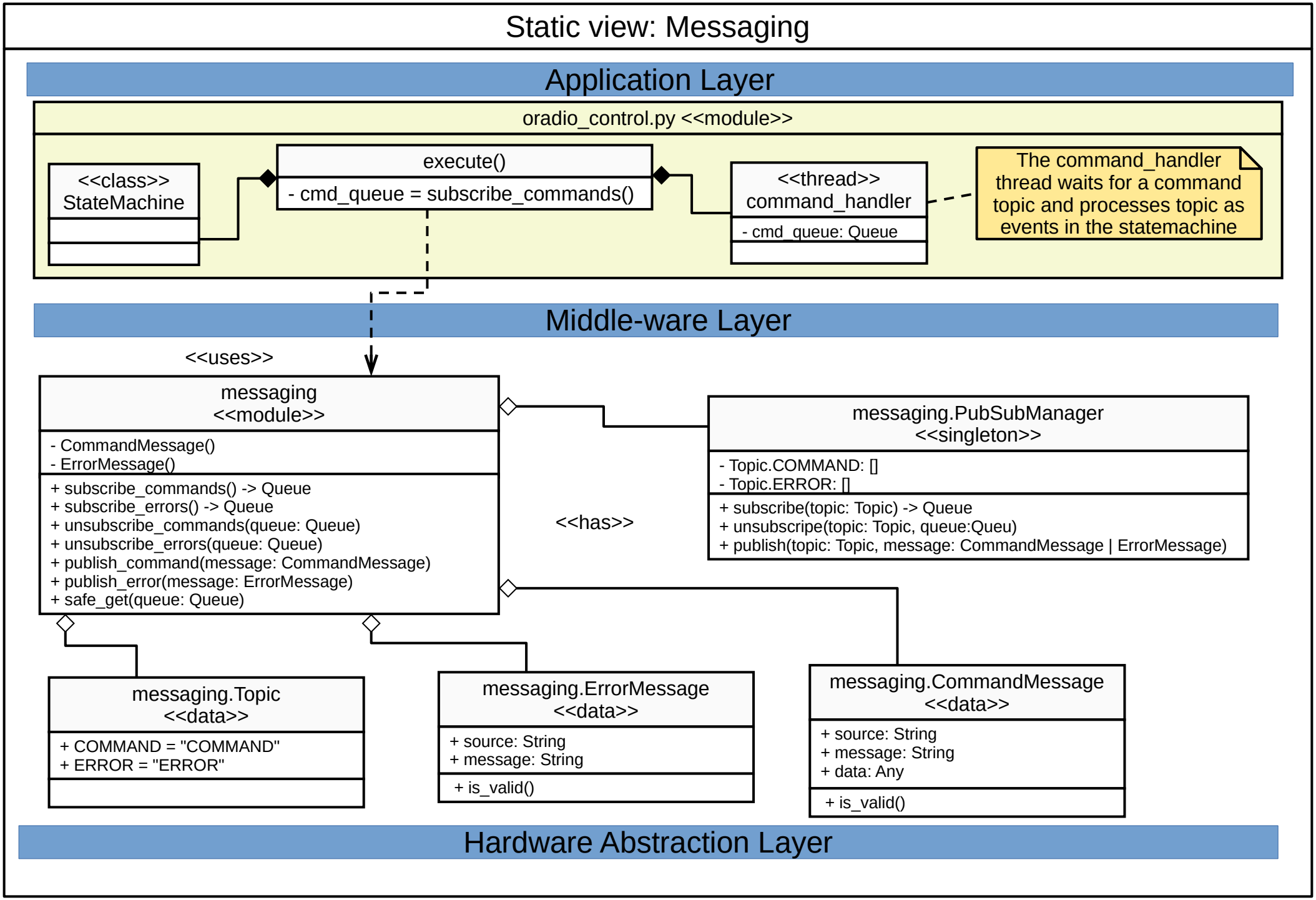

Static view: USB
Application Layer
oradio_control.py <<module>>
execute()
- usb_service = USBService()
Middle-ware Layer
<<uses>>
<<module>>
messaging.py
+ publish_command_message(
message: CommandMessage)
usb_service.USBService
<<class>>
- observer: Observer()
+ init()
+ get_state()
<<module>>
wifi_service.py
+ networkmanager_add(
network:str, password:str)
<<uses>>
<<consists of>>
<<schedules>>
<<singleton>>
usb_service.USBObserver
- observer: Observer
+ on_created(event)
+ on_deleted (event)
<<Library>>
watchdog.observers.
Observer
- USBObserver:
+ schedule()
+ start()
<<uses>>
<<BaseClass>>
FileSystemEventHandler
+ on_created(event)
+ on_deleted(event)
<<is a>
Hardware Abstraction Layer

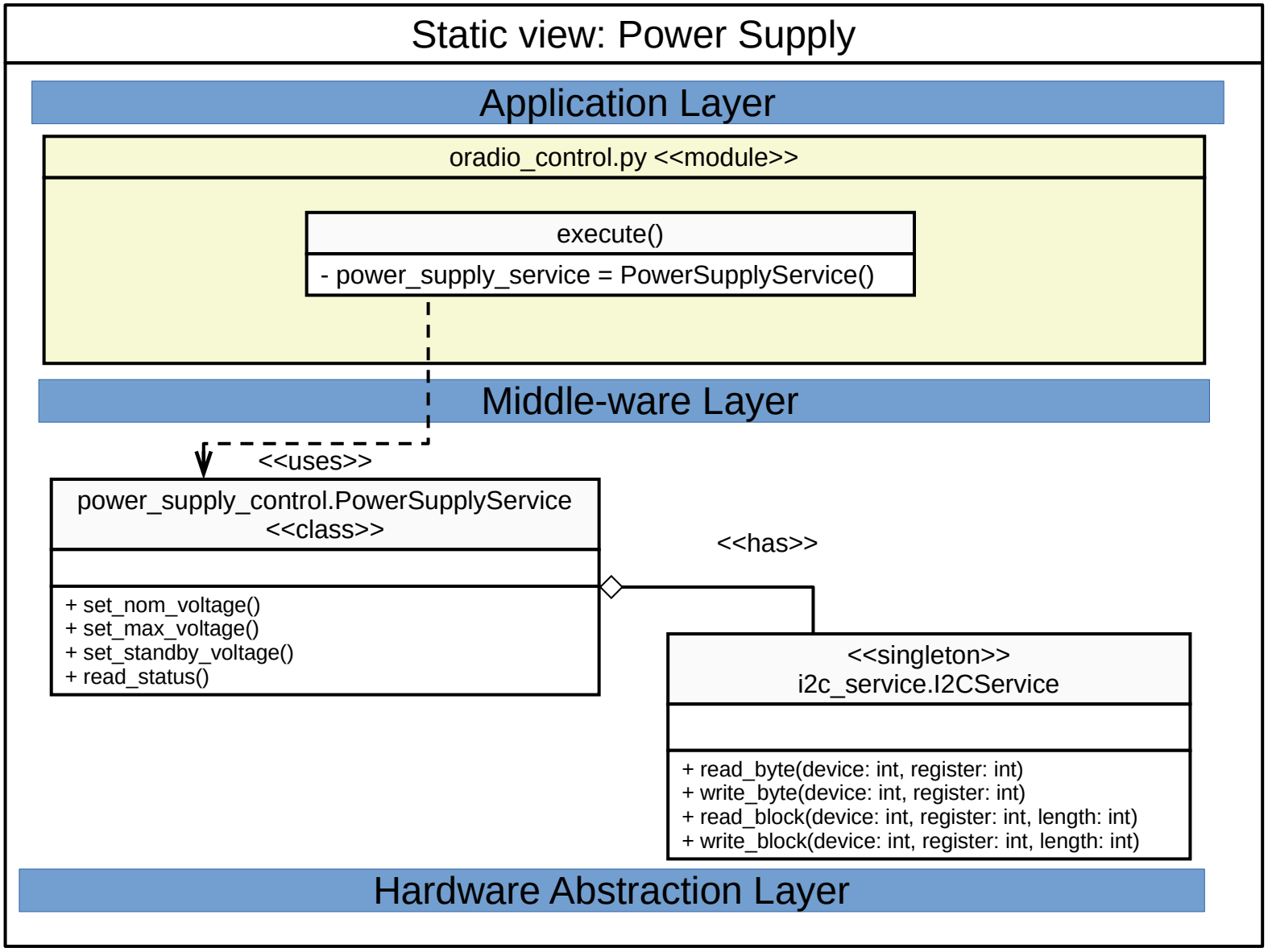
Static view: Volume

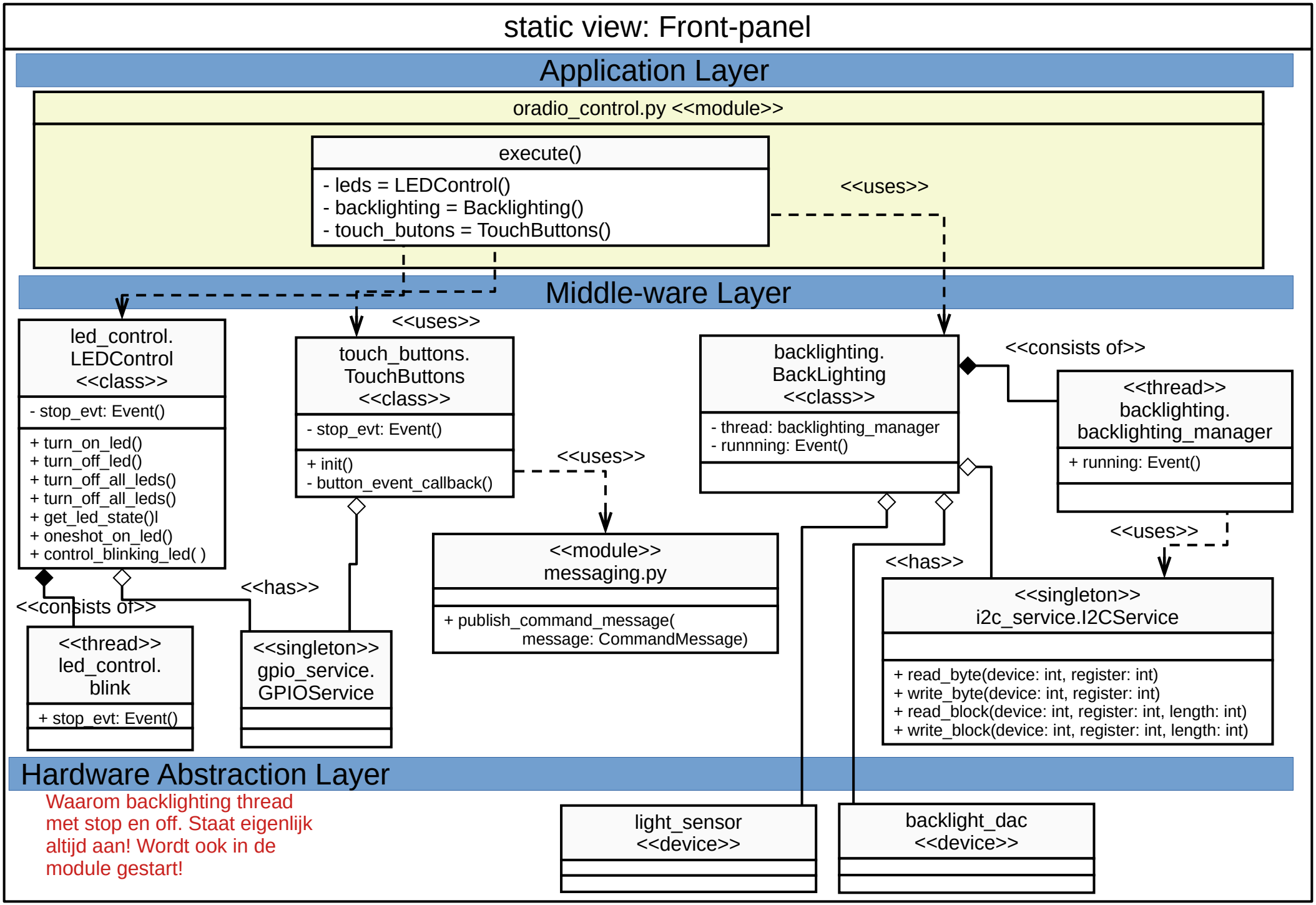


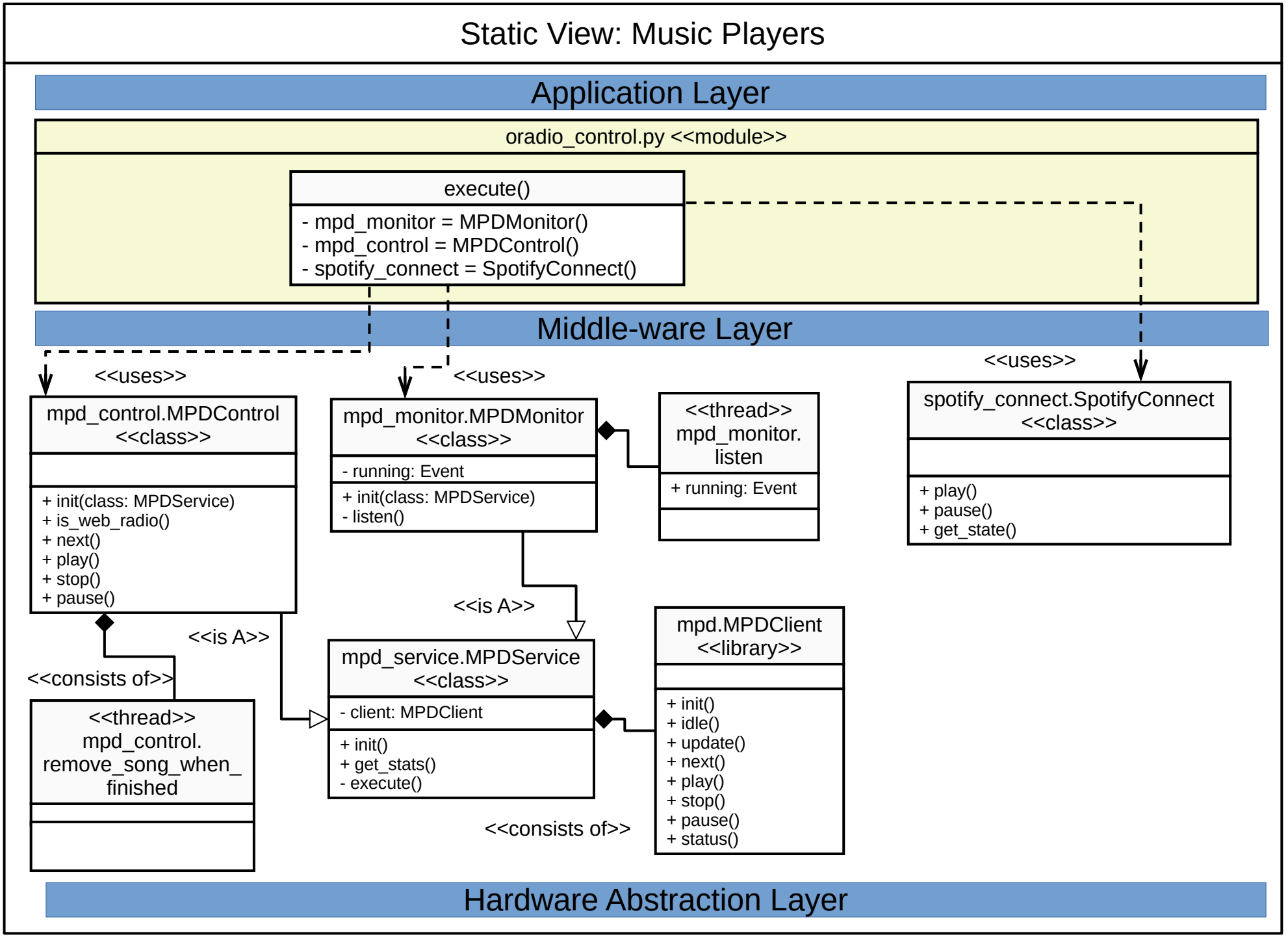

Static view: Error
Application Layer
oradio_control.py <<module>>
execute()
- error_service = ErrorService()
Middle-ware Layer
<<uses>>
error_service.py.ErrorService
<<class>>
- error_queue:subscriber_errors()
- init()
- error_handler()
<<thread>>
error_service.error_handler
<<uses>>
<<module>>
messaging.py
+ ErrorMessage
+ safe_get(error_queue: Queue)
Hardware Abstraction Layer

Static view: Messaging




Static view: Power Supply
Application Layer
oradio_control.py <<module>>
execute()
- power_supply_service = PowerSupplyService()
Middle-ware Layer
<<uses>>
power_supply_control.PowerSupplyService
<<class>>
+ set_nom_voltage()
+ set_max_voltage()
+ set_standby_voltage()
+ read_status()
<<has>>
<<singleton>>
i2c_service.I2CService
+ read_byte(device: int, register: int)
+ write_byte(device: int, register: int)
+ read_block(device: int, register: int, length: int)
+ write_block(device: int, register: int, length: int)
Hardware Abstraction Layer


static view: Front-panel
Application Layer
oradio_control.py <<module>>
execute()
- leds = LEDControl()
- backlighting = Backlighting()
- touch_butons = TouchButtons()
<<uses>>
Middle-ware Layer
led_control.
LEDControl
<<class>>
- stop_evt: Event()
+ turn_on_led()
+ turn_off_led()
+ turn_off_all_leds()
+ turn_off_all_leds()
+ get_led_state()l
+ oneshot_on_led()
+ control_blinking_led( )
<<uses>>
touch_buttons.
TouchButtons
<<class>>
- stop_evt: Event()
+ init()
- button_event_callback()
<<uses>>
<<module>>
messaging.py
+ publish_command_message(
message: CommandMessage)
backlighting.
BackLighting
<<class>>
- thread: backlighting_manager
- runnning: Event()
<<consists of>>
<<thread>>
backlighting.
backlighting_manager
+ running: Event()
<<uses>>
<<consists of>>
<<has>>
<<has>>
<<thread>>
led_control.
blink
+ stop_evt: Event()
<<singleton>>
gpio_service.
GPIOService
<<singleton>>
i2c_service.I2CService
+ read_byte(device: int, register: int)
+ write_byte(device: int, register: int)
+ read_block(device: int, register: int, length: int)
+ write_block(device: int, register: int, length: int)
Hardware Abstraction Layer
Waarom backlighting thread
met stop en off. Staat eigenlijk
altijd aan! Wordt ook in de
module gestart!
light_sensor
<<device>>
backlight_dac
<<device>>


Static View: Music Players
Application Layer
oradio_control.py <<module>>
execute()
- mpd_monitor = MPDMonitor()
- mpd_control = MPDControl()
- spotify_connect = SpotifyConnect()
Middle-ware Layer
<<uses>>
<<uses>>
<<uses>>
mpd_control.MPDControl
<<class>>
+ init(class: MPDService)
+ is_web_radio()
+ next()
+ play()
+ stop()
+ pause()
mpd_monitor.MPDMonitor
<<class>>
- running: Event
+ init(class: MPDService)
- listen()
<<thread>>
mpd_monitor.
listen
+ running: Event
spotify_connect.SpotifyConnect
<<class>>
+ play()
+ pause()
+ get_state()
<<is A>>
<<is A>>
<<consists of>>
<<thread>>
mpd_control.
remove_song_when_
finished
mpd_service.MPDService
<<class>>
- client: MPDClient
+ init()
+ get_stats()
- execute()
mpd.MPDClient
<<library>>
+ init()
+ idle()
+ update()
+ next()
+ play()
+ stop()
+ pause()
+ status()
<<consists of>>
Hardware Abstraction Layer